LAPORAN TUGAS BESAR
IF2211 STRATEGI ALGORITMA
PEMANFAATAN ALGORITMA GREEDY DALAM PEMBUATAN BOT PERMAINAN BATTLECODE 2025

Dipersiapkan oleh:

Kelompok Bonaparte

13524025 - Moh. Hafizh Irham Perdana

13524047 - Muhammad Jordan Ferimeison

13524099 - Muhammad Akmal

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESHA 10, BANDUNG 40132

2025

Daftar Isi

Daftar Isi **2**

Daftar Tabel **3**

A. BAB 1 ..... **4**

A. Latar Belakang 4

B. Deskripsi Permainan 4

C. Komponen Permainan 5

1. Peta 5

2. Robot 5

3. Tower 6

4. Paint dan Chips 7

5. Komunikasi 8

D. Spesifikasi Tugas 8

B. **9**

C. BAB 2 ..... **9**

A. Definisi Algoritma Greedy 9

B. Komponen Algoritma Greedy 10

C. Jenis Algoritma Greedy 12

D. BAB 3 ..... **13**

A. Elemen Algoritma 14

B. Alternatif Solusi 17

E. BAB 4 ..... **24**

A. Implementasi 24

B. Pengujian 24

F. BAB 5 ..... **25**

A. Kesimpulan 25

B. Saran 25

Bab 1 Deskripsi Tugas

A. Latar Belakang

Battlecode adalah kompetisi pemrograman berbasis game strategi *real time* yang diselenggarakan secara tahunan oleh Massachusetts Institute of Technology (MIT). Pada kompetisi ini, setiap pemain diharuskan menulis kode untuk menjalankan bot. Pemain harus mengatur berbagai jenis robot secara strategis dan mengendalikan cara robot-robot tersebut bekerja sama untuk mengalahkan tim lawan. Sebagai peserta, pemain dapat belajar menggunakan *artificial intelligence, path tracking, distributed algorithm*, dan komunikasi untuk membuat antarpemain bersaing sebaik mungkin. Namun, pemain hanya memiliki *computation resource* yang terbatas setiap giliran atau *turn* dalam game, sehingga pemain harus merencanakan strategi, mengoptimalkan, dan mengoordinasikan algoritma robot secara efektif.

Pada Tugas Besar 1 mata kuliah IF2211 Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot Battlecode 2025 yang mengimplementasikan algoritma greedy. Bot yang dibuat akan dipertandingkan satu sama lain antarmahasiswa. Tujuan utama dari tugas besar ini adalah mengimplementasikan konsep algoritma greedy pada game dengan Battlecode 2025 dengan berbagai faktor dan parameter yang kompleks.

1. Deskripsi Permainan

Battlecode 2025 adalah game strategi yang dimainkan dalam grid persegi dua dimensi berukuran hingga 60x60. Dua tim akan saling berhadapan, masing-masing mengendalikan sejumlah robot dan tower. Setiap tim berusaha untuk memenangkan permainan melalui salah satu dari dua cara berikut:

1. Mewarnai lebih dari 70% petak pada peta
2. Menghancurkan seluruh robot dan tower tim lawan

Pergerakan robot dan tower setiap tim akan dihitung dalam satuan ronde. Jika setelah 2000 ronde tidak terdapat tim yang memenuhi kondisi kemenangan tersebut, pemenang ditentukan berdasarkan urutan tie breaker, yaitu persentase area yang telah

diwarnai, jumlah tower yang masih berdiri, total chips(mata uang dalam game), total cat, dan jumlah robot yang masih hidup.

B. Komponen Permainan

1. Peta

Setiap permainan dimainkan pada peta berbentuk grid persegi dua dimensi berukuran antara 20x20 hingga 60x60 petak. Koordinat (0, 0) terdapat di pojok kiri bawah peta dengan koordinat x yang bertambah ke arah Timur(kanan) dan koordinat y yang bertambah ke arah Utara(atas). Peta dijamin bersifat simetris baik melalui rotasi maupun refleksi untuk memastikan tidak terdapat sisi yang diuntungkan. Peta dapat memilki tembok(*walls*) yang tidak dapat dilewati bot atau diwarnai dengan jumlah maksimal 20% dari seluruh peta.

2. Robot

Robot adalah unit utama yang dikendalikan dalam permainan. Setipa robot menjalankan salinan kode yang sama secara independen. Terdapat tiga jenis robot dengan karakteristik sebagai berikut:

Jenis	HP	Max Paint	Production Cost	Attack Radius	Action Cooldown
Soldier	250	200	200 paint + 250 chips	3	10
Mopper	50	100	100 paint + 300 chips	1,41	30
Splasher	150	300	300 paint + 400 chips	2 (AoE)	50

Selain itu, terdapat penalti paint bagi robot apabila paint robot di bawah 50% yaitu peningkatan cooldown pergerakan dan *action*. Apabila paint habis,

robot tidak dapat bergerak atau melakukan *action* dan kehilangan 20 HP per ronde.

3. Tower

Menara atau tower berfungsi sebagai zona *spawn* unit(termasuk pemilihan jenis robot) dan dapat menyerang robot musuh dalam radius tertentu. Setiap tower dapat ditingkatkan menggunakan chips hingga level 3. Tower harus dibangun di atas *ruins* dengan cara mewarnai pola 5x5 di sekitarnya, lalu memanggil prosedur completeTowerPattern(). Setiap tim dapat memiliki maksimal 25 menara dalam permainan. Terdapat tiga jenis tower dengan karakteristik sebagai berikut:

Jenis	**Build Cost**	**Attack Range**	**Attack Strength (Single/AoE)**	**Base Mining Rate**	**Health**
Money Tower lv1	1000 chips	3	20/10	20 chips/turn	1000
Money Tower lv2	2500 chips	3	20/10	30 chips/turn	1500
Money Tower lv3	5000 chips	3	20/10	40 chips/turn	2000
Paint Tower lv1	1000 chips	3	20/10	5 paint/turn	1000

Paint Tower lv2	2500 chips	3	20/10	10 paint/turn	1500
Paint Tower lv3	5000 chips	3	20/10	15 paint/turn	2000
Defense Tower lv1	1000 chips	4	40/20	20 chips/attack	2000
Defense Tower lv2	2500 chips	4	50/25	30 chips/attack	2500
Defense Tower lv3	5000 chips	4	60/35	40 chips/attack	3000

4. Paint dan Chips

Cat atau Paint adalah sumber daya utama yang digunakan untuk mewarnai petak dan mengisi ulang simpanan paint robot. Chips digunakan untuk membangun robot dan towe. Setiap tim memulai permainan dengan 2500 chips. Paint dan Chips masing-masing dihasilkan secara pasif oleh Paint Tower dan Money Tower setiap ronde

5. Komunikasi

Robot dan tower dapat berkomunikasi menggunakan *messsages* yang berisi bilangan bulat 32 bit, dengan batas satu pesan per robot pada setiap ronde.

Komunikasi hanya dapat terjadi jika robot berada dalam jarak 4,47 petak dari menara dan terhubung melalui jalur paint sekutu. Selain *messages*, robot juga dapat menggunakan penanda atau *markers* untuk menandai petak pada peta untuk membentuk pola 5x5 pada ruin.

2. Spesifikasi Tugas

Berikut adalah spesifikasi wajib yang harus dipenuhi dalam pengerjaan tugas besar ini:

1. Membuat tiga bot dalam bahasa Java dengan salah satunya adalah bot utama dan dua sisanya adalah bot alternatif yang mengimplementasikan algoritma greedy untuk memenangkan permainan Battlecode 2025.

2. Setiap bot harus mengimplementasikan strategi greedy yang berbeda dengan heuristik yang berbeda.

3. Bot yang dibuat tidak boleh sama dengan sampel yang diberikan sebagai contoh maupun bot yang telah digunakan dalam kompetisi resmi.

4. Tugas dikerjakan secara berkelompok dengan anggota sebanyak 3 orang.

5. Pengumpulan tugas meliputi source code yang dapat dijelaskan beserta README dan laporan soft copy.

Bab 2
Landasan Teori

3. Definisi Algoritma Greedy

Algoritma greedy adalah suatu metode pemecahan masalah yang membangun solusi langkah demi langkah dengan cara selalu memilih pilihan yang terlihat paling menguntungkan (locally optimal) pada setiap tahap pengambilan keputusan, tanpa mempertimbangkan dampak dari keputusan tersebut terhadap langkah-langkah selanjutnya. Kata "greedy" (serakah) mencerminkan sifat algoritma ini: ia langsung mengambil opsi terbaik yang tersedia saat ini tanpa menunggu atau memeriksa seluruh kemungkinan.

Secara formal, algoritma greedy bekerja dengan prinsip berikut: pada setiap tahap pengambilan keputusan, dari sekumpulan kandidat yang tersedia, dipilih satu kandidat yang memaksimalkan (atau meminimalkan) suatu fungsi heuristik lokal. Keputusan ini bersifat irrevocable, artinya sekali suatu kandidat dipilih, keputusan tersebut tidak akan diubah pada langkah berikutnya

Algoritma greedy tidak selalu menghasilkan solusi optimal secara global. Namun, untuk masalah-masalah tertentu yang memiliki sifat greedy choice property dan optimal substructure, algoritma greedy terbukti menghasilkan solusi optimal. Dua sifat tersebut didefinisikan sebagai berikut:

- Greedy Choice Property: Suatu masalah memiliki sifat ini jika solusi optimal global dapat dibangun dari pilihan-pilihan optimal lokal. Artinya, keputusan greedy terbaik pada setiap langkah pasti merupakan bagian dari setidaknya satu solusi optimal.
- Optimal Substructure: Suatu masalah memiliki sifat ini jika solusi optimal dari masalah tersebut mengandung solusi optimal dari sub-masalahnya. Setelah suatu pilihan greedy dibuat, sisa masalah yang belum terselesaikan membentuk sub-masalah serupa yang lebih kecil.

Pada masalah yang tidak memenuhi kedua sifat di atas, seperti pada sistem permainan Battlecode 2025, algoritma greedy digunakan sebagai pendekatan heuristik yang memberikan solusi yang cukup baik (satisficing solution) dengan biaya komputasi yang sangat rendah mengingat batasan bytecode setiap turn dalam permainan.

4. Komponen Algoritma Greedy

Setiap implementasi algoritma greedy terdiri dari enam elemen fundamental yang saling berkaitan. Pemahaman terhadap keenam elemen ini diperlukan untuk merancang dan menganalisis suatu algoritma greedy secara sistematis. Algoritma greedy melibatkan pencarian sebuah himpunan bagian S dari himpunan kandidat C, yang dalam hal ini S harus memenuhi beberapa kriteria yang ditentukan, yaitu A menyatakan suatu solusi dan S dioptimisasi oleh fungsi objektif.

1. Himpunan Kandidat

   Himpunan kandidat C adalah kumpulan semua elemen yang berpotensi menjadi bagian dari solusi. Pada awal algoritma, seluruh elemen yang mungkin dipilih terdapat dalam himpunan ini. Setiap kali sebuah elemen dipilih (atau dibuang karena tidak layak), ia dihapus dari himpunan kandidat. Algoritma berhenti ketika himpunan kandidat kosong atau ketika solusi sudah terbentuk. Contohnya adalah dalam masalah minimum spanning tree (Kruskal), himpunan kandidat adalah seluruh sisi (edge) pada graf yang belum diproses.

2. Fungsi Seleksi

   Fungsi seleksi adalah inti dari algoritma greedy. Fungsi ini menentukan kandidat mana yang dipilih dari himpunan kandidat pada setiap langkah berdasarkan suatu kriteria heuristik. Pemilihan fungsi seleksi yang tepat menentukan apakah algoritma greedy akan menghasilkan solusi optimal atau tidak.

   Fungsi seleksi yang umum digunakan antara lain:

   a. Pilih kandidat dengan nilai terbesar (*maximization greedy*)
   b. Pilih kandidat dengan nilai terkecil (*minimization greedy*)

c. Pilih kandidat yang paling dekat dengan posisi saat ini (*nearest neighbor greedy*)
d. Pilih kandidat dengan rasio nilai/biaya terbaik (*fractional greedy)*

Contoh fungsi seleksi adalah dalam masalah activity selection, fungsi seleksi memilih aktivitas dengan waktu selesai paling awal karena hal ini memaksimalkan jumlah aktivitas yang dapat dipilih selanjutnya.

3. Fungsi Kelayakan

Fungsi kelayakan memeriksa apakah suatu kandidat yang telah dipilih oleh fungsi seleksi dapat ditambahkan ke dalam himpunan solusi tanpa melanggar batasan atau kendala masalah. Jika kandidat tersebut layak, ia dimasukkan ke solusi. Jika tidak layak, kandidat tersebut dibuang dari himpunan kandidat.

Namun penting untuk dicatat bahwa dalam algoritma greedy murni, kandidat yang tidak layak langsung dibuang tanpa ditinjau kembali. Berbeda dengan backtracking yang dapat membatalkan pilihan sebelumnya jika terbukti tidak menghasilkan solusi.

Contoh: dalam masalah knapsack 0/1, fungsi kelayakan memeriksa apakah berat total barang yang dipilih tidak melebihi kapasitas tas.

4. Fungsi Solusi

Fungsi solusi menentukan kapan himpunan kandidat yang telah terpilih sudah membentuk solusi yang lengkap dan valid. Ketika fungsi solusi mengembalikan nilai true, algoritma berhenti dan himpunan solusi saat itu dinyatakan sebagai jawaban akhir.

Fungsi solusi berbeda dari fungsi kelayakan: fungsi kelayakan memeriksa apakah penambahan satu kandidat baru tidak melanggar batasan, sedangkan fungsi solusi memeriksa apakah keseluruhan himpunan solusi yang sudah terbentuk sudah memenuhi syarat sebagai solusi lengkap.

Contoh: dalam masalah Huffman Coding, fungsi solusi terpenuhi ketika hanya tersisa satu pohon dalam antrian prioritas.

5. Himpunan Solusi

   Himpunan solusi S adalah kumpulan kandidat yang telah dipilih oleh fungsi seleksi dan dinyatakan layak oleh fungsi kelayakan. Himpunan ini dibangun secara bertahap: dimulai dari kosong ($S = \varnothing$), lalu satu per satu elemen ditambahkan hingga fungsi solusi terpenuhi. Himpunan solusi merepresentasikan jawaban akhir dari algoritma greedy. Nilai fungsi objektif dihitung berdasarkan elemen-elemen yang terdapat dalam himpunan solusi ini

6. Fungsi Objektif

   Fungsi objektif mengukur kualitas atau nilai dari solusi yang dihasilkan. Tujuan algoritma greedy adalah memaksimalkan (dalam masalah maximization) atau meminimalkan (dalam masalah minimization) nilai fungsi objektif ini. Fungsi objektif menjadi acuan dalam mengevaluasi seberapa baik solusi yang dihasilkan dibandingkan dengan solusi optimal.

   Selain itu, karena sifat greedy yang hanya mempertimbangkan optimum lokal, nilai fungsi objektif dari solusi greedy tidak selalu sama dengan nilai optimal global, kecuali untuk masalah-masalah yang terbukti memiliki greedy choice property.

5. Jenis Algoritma Greedy

Algoritma greedy dapat dikategorikan berdasarkan jenis masalah yang diselesaikan dan strategi pemilihan yang digunakan. Berikut adalah jenis-jenis utama algoritma greedy beserta karakteristiknya.

a. Greedy berdasarkan nilai terbesar

   Strategi ini memilih kandidat dengan nilai atau manfaat terbesar pada setiap langkah. Strategi ini paling sederhana dan intuitif, cocok untuk masalah di mana elemen dengan nilai terbesar memberikan kontribusi terbesar terhadap fungsi objektif.

b. Greedy berdasarkan jarak terdekat

Strategi ini memilih kandidat yang paling dekat dengan posisi atau keadaan saat ini. Digunakan terutama pada masalah yang melibatkan traversal ruang atau pencarian jalur, di mana jarak merupakan metrik utama yang diminimalkan.

1. Greedy berdasarkan deadline

   Strategi ini memprioritaskan kandidat berdasarkan tenggat waktu (deadline) atau urgensi. Digunakan untuk masalah penjadwalan di mana setiap pekerjaan memiliki batas waktu yang harus dipenuhi agar mendapatkan keuntungan.

2. Greedy berdasarkan rasio

   Strategi ini memilih kandidat berdasarkan rasio antara dua metrik, biasanya nilai terhadap biaya. Strategi ini lebih canggih dari greedy nilai murni karena mempertimbangkan efisiensi setiap kandidat, bukan hanya nilai absolutnya.

3. Greedy berbasis heuristik komposit

   Strategi ini menggabungkan beberapa metrik sekaligus dalam satu fungsi seleksi yang lebih kompleks, dengan pemberian bobot berbeda pada setiap faktor. Digunakan ketika satu metrik tunggal tidak cukup untuk membuat keputusan yang baik.

Bab 3
Aplikasi Strategi Greedy

1. Elemen Algoritma

Sebelum merancang strategi greedy untuk bot Battlecode 2025, perlu dilakukan pemetaan persoalan ke dalam enam elemen algoritma greedy yang telah dibahas pada Bab 2. Pemetaan ini memastikan bahwa implementasi yang dirancang memiliki landasan teoritis yang kokoh dan dapat dianalisis secara sistematis.

1. Himpunan Kandidat

Himpunan kandidat dalam Battlecode 2025 adalah kumpulan seluruh aksi yang dapat dilakukan oleh setiap robot pada setiap giliran. Karena setiap robot hanya dapat melihat lingkungan dalam radius sensing terbatas (distSq ≤ 20, sekitar 4,47 petak), himpunan kandidat bersifat lokal dan berbeda untuk setiap robot di setiap giliran.

Secara formal, himpunan kandidat untuk robot r pada ronde t didefinisikan sebagai:

$$C(r,t) \;=\; C_{tile}(r,t) \cup C_{robot}(r,t) \cup C_{ruin}(r,t) \cup C_{dir}(r,t)$$

di mana:

- $C_{tile}(r,t)$ adalah semua petak *visible* dalam radius *sensing* yang dapat diwarnai atau di-*mop*.
- $C_{robot}(r,t)$ adalah semua robot musuh *visible* yang dapat diserang dan robot sekuru yang dapat menerima cat.
- $C_{ruin}(r,t)$ adalah semua *ruin visible* yang belum dibangun menjadi tower.

- $C_{dir}(r, t)$ adalah semua arah gerakan yang valid dari posisi saat ini.

2. Himpunan Solusi

Himpunan solusi adalah rangkaian keputusan yang diambil oleh seluruh robot tim sepanjang permainan berlangsung (maksimal 2000 ronde/*turn*). Karena setiap robot beroperasi secara independen, himpunan solusi tim merupakan gabungan dari keputusan seluruh robot:

$S = \cup S(r, t)$ untuk semua robot r dan ronde t

Setiap elemen dari $S(r, t)$ merupakan keputusan tunggal yang diambil robot r pada ronde t. Sebagai contoh, bergerak, menyerang, mewarnai/*painting*, ataupun tidak beraksi sama sekali.

3. Fungsi Seleksi

Fungsi seleksi akan berbeda untuk setiap unit dalam game. Deskripsi untuk masing-masing unit adalah sebagai berikut,

Jenis Unit	**Fungsi Seleksi**	**Heuristik**
Soldier	Pilih petak tidak berwarna terdekat	argmin distSq(myPos, tile) untuk tile tidak berwarna
Splasher	Pilih pusat AoE dengan skor tertinggi	argmax score(c) = Σ (1 + isEnemy(t)) untuk tile t dalam AoE(c)
Mopper	Pilih petak *paint* musuh terdekat	argmin distSq(myPos, tile) untuk tile *paint* musuh
Tower	Serang musuh dengan HP terendah & spawn unit dengan jumlah terendah	argmin HP(enemy); argmin count(unitType)

4. Fungsi Kelayakan

Fungsi kelayakan memastikan bahwa kandidat yang dipilih oleh fungsi seleksi dapat dieksekusi tanpa melanggar batasan dalam game. Batasan-batasan yang diperiksa sebelum unit melakukan aksi adalah sebagai berikut,

Batasan	**Fungsi Kelayakan**	**Penggunaan**
Action cooldown	Hanya menyerang/mewarnai jika action cooldown < 10	`rc.isActionReady()` dan `rc.canAttack(loc)`
Movement cooldown	Hanya bergerak jika movement cooldown < 10	`rc.isMovementReady()` dan `rc.canMove(dir)`
Paint	Refuel jika cat < 30% kapasitas maksimum	`rc.getPaint() < maxPaint * 0.30`
Attack radius	Target harus dalam radius serangan robot	`rc.canAttack(targetLoc)`
Passable tile	Petak tujuan harus dapat dilewati, bukan wall/ruin	`tile.isPassable() && !tile.hasRuin()`
Paint transfer radius	Target transfer harus dalam radius 1.41 tile	`rc.canTransferPaint(targetLoc, amount)`

5. Fungsi Solusi

Fungsi solusi menentukan tim manakah yang menang dan kapan game berakhir. Terdapat tiga kondisi berakhirnya game yaitu,

- **K**ondisi Menang: Tim berhasil mewarnai lebih dari 70% tile yang dapat diwarnai di seluruh peta. Bot memaksimalkan peluang ini dengan memprioritaskan ekspansi wilayah secara agresif.
- Kondisi Menang Alternatif: Tim berhasil menghancurkan seluruh robot dan tower milik tim lawan. Kondisi ini dicapai secara tidak langsung melalui mekanisme serangan tower dan soldier.
- Kondisi Tiebreaker: Jika permainan mencapai 2000 ronde tanpa pemenang, tim dengan luas area terwarnai lebih besar dan *resource* paling banyak dinyatakan menang. Bot yang secara konsisten mewarnai lebih banyak petak per ronde akan unggul dalam tiebreaker ini.

6. Fungsi Objektif

Fungsi objektif utama yang ingin dimaksimalkan oleh bot ini adalah persentase petak terwarnai milik tim terhadap total petak yang dapat diwarnai di peta:

$$f_0(S) = \text{(painted ally area)/(total painted area)} \times 100\%$$

Target dari fungsi objektif adalah $>$70% untuk memenangkan game tanpa tiebreaker.

2. Alternatif Solusi

1. Solusi Utama

Masalah pengendalian agen di Battlecode 2025 dapat dipetakan ke elemen formal dari algoritma greedy. Setiap robot beroperasi dengan penglihatan terbatas (radius sensing) dan harus membuat keputusan yang optimal secara lokal yang berkontribusi pada dominasi peta secara keseluruhan.

Candidate Set (C): Kumpulan semua aksi valid yang dapat diambil robot tertentu pada ronde spesifik t. Karena penglihatan terbatas, himpunan ini sangat terlokalisasi. Ini mencakup:

- Tile yang terlihat untuk dicat atau dibersihkan (mop).
- Musuh yang terlihat untuk diserang atau sekutu untuk didukung/diisi ulang.
- Reruntuhan yang terlihat dan belum dibangun.
- Arah pergerakan yang valid.

Solution Set: Kombinasi dari semua aksi independen yang dijalankan oleh unit tim sepanjang maksimum 2000 ronde ($S = \bigcup S(r, t)$).

Selection Function: Aturan heuristik yang digunakan untuk memilih kandidat lokal "terbaik".

- Soldier: Memprioritaskan tile yang paling dekat dengan posisinya dan musuh dengan HP terendah untuk menjamin kill
- Splasher: Menargetkan pusat AoE yang menjamin jumlah maksimum tile/robot musuh yang dicat per aksi
- Mopper: Menargetkan cat musuh/robot musuh terdekat
- Tower: Menyerang musuh dengan HP terendah dan menghasilkan unit untuk mempertahankan rasio 3:1:1 (Soldier:Mopper:Splasher).
- Feasibility Function: Menyaring aksi yang tidak mungkin dilakukan berdasarkan batasan engine game (misalnya memeriksa `rc.isActionReady()`, memverifikasi `rc.getPaint() >= cost`, dan memastikan peta dapat dilewati).
- Objective Function: Metrik yang dimaksimalkan untuk mengamankan kemenangan. Tujuan utama adalah 70% cat untuk kemenangan.
- Solution Function: Kondisi yang pada akhirnya menyatakan kemenangan. Game memeriksa apakah tim menguasai >70% area peta, telah menghancurkan semua lawan, atau memiliki keunggulan tiebreaker setelah 2000 ronde.

2. Solusi Alternatif 1

Strategi greedy yang diimplementasikan pada setiap tipe unit dalam bot fishybot mencakup urutan prioritas, fungsi seleksi, dan mekanisme keputusan yang digunakan.

a. Tower, unit ini menjalankan tiga fungsi utama setiap giliran: menyerang musuh, spawn unit, dan menyebarkan informasi musuh. Setiap fungsi menggunakan pendekatan greedy yang berbeda.

Prioritas	**Deskripsi**
(I) Attack	Tower dapat melakukan dua jenis serangan setiap turn yaitu, 1. Single Attack dengan menyerang musuh dengan HP terendah terlebih dahulu (greedy kill) untuk memaksimalkan probabilitas eliminasi musuh per turn. 2. AoE Attack dengan menyerang posisi dengan kepadatan musuh tertinggi untuk memaksimalkan total damage per giliran.
(II) Spawn	Tower akan memunculkan robot dengan rasio soldier:mopper:splasher = 3:1:1 untuk mengoptimalkan penggunaan chips.
(III) Broadcast	Jika terdapat musuh terdeteksi, tower mem-broadcast koordinat musuh pertama sebagai integer 32-bit ke tower sekutu lain dalam radius √80. Robot membaca broadcast ini via readBroadcasts() dan menyimpan ke knownEnemyLocation.

b. Soldier, unit ini merupakan unit utama yang bertanggung jawab mewarnai peta dan membangun tower. Ia menjalankan priority waterfall 6 level dengan greedy decision di setiap level.

Prioritas	**Deskripsi**
(I) Upgrade	Soldier melakukan upgrade tower pertama yang terlihat dalam distSq ≤ 2. Tidak memerlukan action cooldown.
(II) Refuel	Soldier akan kembali ke paint tower terdekat apabila paint < 30%. Jika tidak ada navigasi ke savedTowerLocation atau eksplorasi.
(III) Build	Soldier mencari ruin terdekat yang belum diakuisisi menggunakan findBestRuin() kemudian melakukan *commit* untuk melengkapi *pattern* hingga tower dapat terbentuk. Jenis tower yang dibangun didasarkan atas prioritas paint, money, lalu defense.
(IV) Attack	Soldier menyerang musuh terlemah, yaitu yang memilki HP terkecil. Hanya dilakukan jika isActionReady(). knownEnemyLocation dari broadcast tower.
(V) Paint	Soldier mewarnai tile unpainted terdekat dengan kriteria non-ally, passable, dan non-ruin.
(VI) Explore	Jika tidak ada tile unpainted visible soldier akan bergerak ke pusat peta, atau siklus 8 waypoint (4 corner + 4 edge). Robot menyebar secara alami sejak

	spawn karena setiap robot mulai dari waypoint berbeda.

c. Mopper, unit ini berperan sebagai unit pembersih paint musuh sekaligus support refuel. Priority waterfall mopper terdiri dari 6 level

Prioritas	**Deskripsi**
(I) Self Refuel	Mopper akan kembali ke paint tower terdekat apabila paint < 30%.
(II) Ally Refuel	Jika ada soldier dengan cat < 50: transfer min(cat mopper − RESERVE, 40) ke soldier tersebut.
(III) Swing	Melakukan swing pada arah kardinal yang memiliki lebih dari 2 robot musuh sejajar.
(IV) Erase	Menghapus tile dengan paint musuh terdekat. Terdapat paintUnderfoot() yang mencegah mopper terkena double drain di tile musuh tempatnya berdiri.
(V) Follow	Mengikuti soldier terdekat jika tidak ada paint musuh terlihat. Mopper selalu berada dekat soldier untuk siap memberikan cat saat dibutuhkan.
(VI) Explore	Bergerak dengan siklus 8 waypoint. paintUnderfoot() selalu dipanggil di akhir turn.

d. Splasher, unit ini menggunakan serangan AoE yang mewarnai banyak tile sekaligus. Heuristiknya berfokus pada memaksimalkan jumlah tile yang terwarnai per satu aksi

Prioritas	**Deskripsi**
(I) Refuel	Splasher akan kembali ke paint tower terdekat apabila paint < 30%.
(II) Attack	Splasher akan menyerang pada tile dengan jumlah musuh terbanyak atau paint kosong terluas.
(III) Move	Splasher akan bergerak ke tile paling strategis menggunakan bestMoveTarget untuk memaksimalkan attack.
(IV) Explore	Splasher bergerak dengan siklus 8 waypoint

3. Solusi alternatif 2

Strategi greedy yang diimplementasikan pada birdiebot mencakup,

a. Tower

Prioritas	**Deskripsi**
(I) Attack	Tower akan menyerang 2 musuh pertama yang ditemui (dengan single dan AOE attack).
(II) Spawn	Tower akan memunculkan robot dengan heuristik;

	Mopper jika dibutuhkan (dengan cara cek pesan) → Splasher jika memungkinkan → Soldier jika memingkinkan
(III) Upgrade	Tower akan meningkatkan diri sendiri jika memungkinkan

b. Soldier

i. Soldier akan mengecek jumlah paint. Jika rencah, cari Tower terdekat dan isi paint.

ii. Jika menemukan ruin, coba untuk membangun tower. Tower diprioritaskan dari Paint, Money, dan Defense.

iii. Setelah itu, cari satu arah yang ingin dituju melalui fungsi antara. Berjalan menuju arah tersebut.

iv. Selalu paint empty tiles jika memungkinkan.

Prioritas	**Deskripsi**
(I) Retreat	Soldier akan mengisi paint jika kurang dari batas yang ditentukan. Soldier akan pergi ke tower terdekat.
(II) Tower	Soldier akan membangun Tower jika memungkinkan. Urutan pembangunan adalah Defense → Money → Paint dengan rasio yang ditentukan (1:2:3) (*→ untuk priority)

(III) Movement	Soldier akan bergerak ke arah yang ditentukan oleh fungsi bantuan (arah akan berbentuk koordinat a,b jika fungsi berhasil atau random jika fungsi gagal)
(IV) Paint	Soldier akan mengecat petak kosong yang ditemui.

c. Mopper

Prioritas	**Deskripsi**
(I) Menghapus Petak	Mopper akan menghapus petak musuh jika memungkinkan.
(II) Menyerang musuh	Mopper akan menyerang musuh jika memungkinkan
(III) Support	Memberi paint ke allies jika memungkinkan
(IV) Movement	Bergerak ke arah musuh yang ditemukan atau bergerak ke arah acak (tergantung menemukan musuh atau tidak).

d. Splasher

Prioritas	**Deskripsi**
(I) Attack	Splasher akan mencari petak dengan adjacent tiles (musuh → kosong → allies) terbanyak. (*→ untuk

	priority)
(II) Movement	Bergerak ke petak musuh terdekat → bergerak ke petak yang belum diwarnai terdekat → bergerak secara acak. (*→ untuk priority)

Bab 4 Implementasi dan Pengujian

1. Implementasi

1. Solusi Utama (BirdieBot)

Memetakan proses pengambilan keputusan dari birdiebot (Alternative Solution 1) ke dalam enam elemen teoretis strategi greedy:

- **Candidate Set:** Kumpulan aksi yang valid untuk tipe unit tertentu, ditentukan oleh field of view lokal (distSq <= 20). Secara formal,

- $C(r,t)=C_{tile}(r,t) \cup C_{robot}(r,t) \cup C_{ruin}(r,t) \cup C_{dir}(r,t)$
- $C_{tile}(r,t)=C$
- $(r,t) \cup C$
- $C_{robot}(r,t)=C$
- $(r,t) \cup C$
- $C_{ruin}(r,t)=C$
- $(r,t) \cup C$
- $C_{dir}(r,t)=C$
- (r,t).
- **Solution Set:** Kumpulan keputusan agregat yang diambil oleh semua bot dalam tim sepanjang maksimum 2000 ronde permainan.
- **Selection Function:** Menentukan aksi lokal optimal melalui sistem **priority waterfall**:
    - **Soldier:** Secara greedy memilih tower terdekat untuk mengambil paint ketika rendah. Memilih ruin terdekat untuk memenuhi rasio tower **1:2:3 (Defense:Money:Paint)**. Menargetkan tile kosong terdekat untuk diwarnai.
    - **Mopper:** Menghitung arah kardinal yang dapat mengenai jumlah musuh terbanyak menggunakan serangan AoE swing.
    - **Splasher:** Menargetkan pusat splash dengan skor maksimum menggunakan heuristik
    - $\sum(\text{isEmpty}\times 2+\text{isEnemy}\times 3)$
    - $\sum(\text{isEmpty}\times 2+\text{isEnemy}\times 3)$ dalam AoE ukuran 4.
    - **Tower:** Secara greedy menyerang dua musuh pertama yang terdeteksi secara berurutan. Memunculkan Mopper secara darurat jika permintaan broadcast

    - ≥ 2
    - ≥ 2.
- **Feasibility Function:** Memastikan aksi sesuai dengan batasan mekanisme game (isActionReady(), canAttack(), canBuildRobot(), canMove()).
- **Solution Function:** Kondisi kemenangan terjadi ketika tim menguasai >70% area peta atau menghancurkan semua tower musuh.
- **Objective Function:** Maksimisasi lokal terhadap skor cakupan paint:
- Ally AreaTotal Colorable Area×100%
    - Total Colorable Area
        - Ally Area
            -
- ×100%

  sambil menghancurkan unit lawan untuk meningkatkan surplus chip.

Analisis Efisiensi dan Efektivitas

- **Efisiensi:**
  Heuristik greedy berjalan dalam kompleksitas kira-kira
- O(N)
- $O(N)$ terhadap jumlah tile/unit dalam radius sensing robot. Alih-alih melakukan pathfinding global pada seluruh peta, keputusan dibuat secara lokal sehingga menghemat bytecode. Targeting Splash menggunakan nested loop pada tile yang terlihat, menjaga batas komputasi tetap rendah sambil memaksimalkan efek serangan area.
- **Efektivitas:**
  Pengendalian peta dilakukan secara terdesentralisasi dengan pembagian tugas:
    - **Soldier** menangani pengecatan dan pembangunan tower.
    - **Mopper** membersihkan area yang dikuasai musuh.
    - **Splasher** bertindak sebagai unit serangan area untuk merebut wilayah dengan cepat.

Strategi Greedy yang Dipilih, Alasan, dan Pertimbangan

Strategi yang diimplementasikan adalah **Adaptive Priority Waterfall**, yang sangat dipengaruhi oleh kondisi paint dan sistem broadcasting.

- **Alasan:**
  Pendekatan ini menyeimbangkan kelangsungan hidup unit dengan ekspansi agresif. Dengan mundur ketika paint turun di bawah 30%, bot dapat mempertahankan sumber daya. Sistem

Mopper-on-demand mencegah terlalu banyak Mopper dibuat, sehingga chip dapat dialokasikan untuk Splasher atau Soldier hingga wilayah musuh benar-benar terbentuk.

- **Pertimbangan:**
  Karena fungsi seleksi hanya mengevaluasi variabel *terdekat* atau *skor tertinggi* tanpa melihat masa depan, bot dapat terjebak pada **local minima** (misalnya terus berpatroli di tepi peta jika sudut peta sulit dijangkau).

2. Implementation Design

Pseudocode

```
// TOWER SCRIPT

procedure runTower(rc):

    Read incoming messages -> update mopperDemand

    For each enemy in vision:

        if canAttack -> attack(enemy) (Max 2 attacks)


    If actionReady and canSpawn:

        if mopperDemand >= 2 and chips >= 300 and paint >= 100: spawn(MOPPER)

        else if chips > 1500 and paint >= 300: spawn(SPLASHER)

        else if chips >= 250 and paint >= 200: spawn(SOLDIER)


    If canUpgrade(myLoc) -> upgradeTower(myLoc)


// SOLDIER SCRIPT

procedure runSoldier(rc):

    if paint < 30% -> retreating = true
```

```
    if paint > 80% -> retreating = false


    notifyTowerIfEnemyOrRuinFound()


    if retreating:
        tower = nearestTower()
        moveTo(tower) OR randomMove()
        return


    ruin = nearestRuin()
    if ruin is close (distSq <= 8):
            buildTower(rc, ruin) // Prioritizes 1:3:2 (Def:Paint:Mon) greedily based on past
builds


    target = nearestUnpainted() OR cyclicEdgeMapBounds()
    moveTo(target)


    bestTile = nearestUnpaintedPassableTile()
    if bestTile != null -> attack(bestTile) // Paint it


// MOPPER SCRIPT
procedure runMopper(rc):
    if actionReady: aggressivelySwingAtMaxEnemies()
```

```
    if actionReady: mopNearestEnemyPaint()

    givePaintToNearbySoldiersLessThan40Percent()

    target = farthestEnemyPaintFromSpawns()
    if target != null -> moveTo(target)
    else -> randomMove()

// SPLASHER SCRIPT
procedure runSplasher(rc):
    if actionReady:
        bestLoc = getHighestSplasherScoreLoc() // empty=2 points, enemy=3 points
        attack(bestLoc)

    enemy = nearestEnemyPaint()
    unpainted = nearestUnpainted()

    if enemy -> moveTo(enemy)
    else if unpainted -> moveTo(unpainted)
    else -> moveRandom()
```

Penjelasan Struktur Data dan Fungsi

- **MapInfo[] / RobotInfo[]:** Array inti yang disediakan oleh engine Battlecode untuk membaca kondisi lingkungan melalui fungsi sensing seperti senseNearbyMapInfos().
- **Static Global State Variables:** Variabel primitif sederhana untuk melacak perubahan perilaku (builtMoney, builtPaint, builtDefense, retreating). Disimpan sebagai variabel statis agar state tetap bertahan antar turn dengan overhead memori minimal.
- **Messaging System:** Integer yang dikodekan dikirim sebagai array 24-bit menuju koordinat tower tertentu (MSG_ENEMY_PAINT, MSG_REQUEST_MOPPER, MSG_RUIN_FOUND). Digunakan oleh Soldier untuk memberi informasi situasional kepada Tower.
- **chooseTowerType():** Mengimplementasikan pemilihan greedy dengan memeriksa rasio tower yang sudah dibangun menggunakan konstanta rasio (PAINT_PER_DEFENSE = 2, PAINT_PER_MONEY = 3).
- **agressiveSwing() / bestSplashTarget():** Fungsi seleksi berbasis skor yang menelusuri data spasial lokal menggunakan iterasi koordinat untuk menghitung koefisien kerusakan area maksimum.

2. Solusi Alternatif 1 (Fishy Bot)

Fishybot (Solusi Utama) vs. **Birdiebot (Alternatif 1)**

Fishybot sangat bergantung pada algoritma priority waterfall 6 tingkat. Efisiensinya berasal dari pemindaian linear

$O(N)$ pada array lokal (misalnya melakukan loop melalui robot yang terlihat untuk menemukan HP terendah). Ini secara efektif menjamin penyebaran sumber daya maksimum dan penolakan area yang cepat dengan secara eksplisit menangani pengisian ulang cat ($\leq$30% memicu kembali).

Bot Utama Birdiebot kurang terstruktur dan bergantung pada pemeriksaan berurutan dasar serta pergerakan acak. Walaupun secara komputasi sangat murah (sangat efisien), efektivitasnya dalam strategi makro kurang karena pergerakan acak ($O(1)$ pilihan arah) menyebabkan cakupan unit yang tumpang tindih dan distribusi cat yang tidak efisien.

Strategi Greedy yang Dipilih

Pemilihan: Kita akan mengimplementasikan **Fishybot Priority Waterfall Strategy**.

Alasan: Model waterfall memastikan unit tidak membuang aksi pada pilihan yang kurang optimal. Dengan mendefinisikan kondisi secara eksplisit (misalnya (I) Upgrade -> (II) Refuel -> (III) Build -> (IV) Attack -> (V) Paint -> (VI) Explore), bot secara

konsisten memprioritaskan struktur dan kelangsungan hidup dibanding agresi buta. Dalam Battlecode, pemanfaatan sumber daya (paint/money) sangat penting, dan rasio unit ketat Fishybot serta broadcast terlokalisasi 80 derajat memberikan ekonomi makro yang jauh lebih unggul dibanding bot alternatif yang acak.

2. Desain Implementasi

2.1 Pseudocode untuk Strategi Prioritas Terpilih (Unit Soldier)

Soldier adalah tulang punggung strategi. Ia mengiterasi prioritas waterfall-nya, dan bertindak pada kondisi valid pertama.

```
procedure executeSoldierTurn(RobotController rc):
    // Priority 1: Upgrade Towers
    if exists visible allied tower in distSq <= 2 and canUpgrade():
        upgradeNearestTower()
        return

    // Priority 2: Refuel
    if rc.getPaint() < (rc.getMaxPaint() * 0.3):
        loc = findNearestAlliedTower()
        moveTowards(loc)
        return

    // Priority 3: Build
    ruin = findNearestUnbuiltRuin()
```

```
if ruin exists:
    if nextTo(ruin) and canBuild():
        buildTowerConditioned(Money > Defense > Paint)
    else:
        moveTowards(ruin)
    return

// Priority 4: Attack
enemies = rc.senseNearbyEnemies()
if enemies is not empty and rc.isActionReady():
    target = min(enemies, key=hp) // Greedy kill
    rc.attack(target)
    // Note: Do not return, can still move

// Priority 5: Paint
if rc.isActionReady():
    bestTile = getNearestUnpaintedTile()
    if bestTile != null:
        rc.attack(bestTile) // Paint it

// Priority 6: Explore
```

```
if rc.isMovementReady():
    if knownEnemyLocation is fresh:
        moveTowards(knownEnemyLocation)
    else:
        moveTowards(getNextWaypoint())
```

2.2 Struktur Data dan Prosedur yang Digunakan

3. **State Variables / Struktur Data:**
    a. MapLocation[] waypoints: Sebuah array dengan panjang 8 yang berisi koordinat sudut dan tepi untuk mensikluskan rute eksplorasi.
    b. int knownEnemyLocation: Sebuah integer 32-bit yang disiarkan oleh tower, didekodekan menjadi koordinat X dan Y yang digunakan di seluruh unit.
4. **Fungsi & Prosedur Inti:**
    a. findBestRuin(RobotInfo[] ruins): Mengevaluasi reruntuhan di sekitar dan mengembalikan reruntuhan terdekat yang belum diambil menggunakan perhitungan jarak minimum.
    b. paintUnderfoot(): Rutinitas keamanan untuk Mopper yang dijalankan di akhir giliran untuk memastikan ia langsung membersihkan tile musuh yang kebetulan diinjaknya, mencegah penalti pengurasan cat ganda.
    c. buildTowerConditioned(): Sebuah prosedur state-machine yang digunakan ketika berada di sebelah reruntuhan. Memeriksa jumlah tower global saat ini dan membangun tower dengan mempertahankan rasio tertentu (misalnya 1 Defense : 2 Money : 3 Paint).
    d. getNearestUnpaintedTile(): Memeriksa array
    e. Ctile(r,t)

f. *C*
g. tile
h.
i. (*r*,*t*) yang disediakan oleh metode rc.senseNearbyMapInfos() dan mengembalikan target dengan kondisi !isAlly() dan isPassable().

5. Solusi Alternatif 2 (Spammer)

Berdasarkan draft kami, kami mempertimbangkan cascade prioritas yang lebih kompleks (seperti **6-level priority waterfall** untuk Soldier pada Fishybot). Arsitektur tersebut sangat efisien untuk manajemen makro di late-game dan dominasi tiebreaker karena secara sistematis menghilangkan wilayah kosong dan secara agresif mengelola ekonomi paint.

Namun, waterfall kompleks memiliki biaya bytecode yang lebih tinggi serta agresivitas awal permainan yang lebih rendah. Sebaliknya, strategi **Spammer** menggunakan footprint algoritmik yang sangat ringan. Heuristik greedy-nya pada dasarnya adalah eksekusi aksi greedy: *"Ambil aksi disruptif pertama yang tersedia tanpa analisis berlebihan."*

- **Effectiveness**: Sangat efektif dalam membanjiri lawan dengan jumlah unit besar dan respons cepat, serta menghindari perhitungan pathing yang kompleks.
- **Efficiency**: Sangat efisien dari sisi pemrosesan bytecode karena hanya menggunakan array sederhana, queue primitif, dan logika fallback acak.

Strategi Greedy yang Dipilih (Spammer)

Strategi yang diimplementasikan memprioritaskan produksi unit besar-besaran dan agresi acak berkelanjutan dibandingkan manajemen ekonomi yang kompleks.

- **Tower** secara greedy memproduksi **Soldier** dan **Mopper** dengan rasio kira-kira **3:2** setiap kali cooldown siap dan terdapat ruang kosong di sekitar. Jika unit yang

diinginkan tidak dapat dibangun, tower akan fallback ke spawn Soldier. Jika ada musuh di sekitar, tower akan langsung menyerang.

- **Soldier** mempertahankan state minimal menggunakan queue primitif. Mereka secara greedy menyerang struktur musuh terlebih dahulu atau mengecat tile kosong di sekitar. Untuk menghindari loop pergerakan, mereka menggunakan fallback acak 50% untuk bergerak tanpa pola. Alih-alih menghitung distSq secara kompleks, mereka mengevaluasi langsung hasil dari senseNearbyRobots() dan senseNearbyMapInfos() lalu berinteraksi dengan target valid pertama yang ditemukan.

2. Desain Implementasi

Eksekusi Tower:

```
procedure Tower.Play()
    if not isActionReady() return
    tentukan target tile menggunakan delta acak di sekitar
    tentukan robotType yang akan dibangun (target rasio 3 Soldier : 2 Mopper)

    if canBuildRobot(robotType, target)
        buildRobot(robotType, target)
        update counters
        return
    else if canBuildRobot(SOLDIER, target)
        buildRobot(SOLDIER, target)
        update soldier counter
        return

    for enemy in senseNearbyRobots(enemies)
```

```
    if canAttack(enemy.location)
        attack(enemy.location)
        return
```

Eksekusi Soldier:

```
procedure Soldier.Play()
  if paint == 0
    disintegrate()
    return

  updateRobotTargets() // Menambahkan struktur/tower musuh ke queue robotTargets
  if tileTargets belum penuh
        updateTileTargets() // Menambahkan tile kosong yang dapat dicat ke queue tileTargets

  if not isActionReady() return

  if randomBoolean() // peluang 50%
    wander()
    clear queues
    return

  target = dequeue dari robotTargets ATAU tileTargets
  if target valid dan canAttack(target)
    attack(target)
    return
  else
```

```
wander()
clear queues
```

Struktur Data, Fungsi, dan Prosedur

- **Struktur Data (spammer/Queue.java)**: Alih-alih menggunakan Java Collections yang mahal atau melakukan iterasi array besar setiap turn, Soldier menggunakan **Queue berbasis array terbatas Queue<MapLocation> berukuran 8**. Ini membatasi memori hanya pada lingkungan sekitar, sehingga mengurangi latency. Satu queue menyimpan robotTargets (khususnya tower musuh) dan queue lainnya menyimpan tileTargets (tile kosong).
- **updateRobotTargets()**: Melakukan sensing robot di sekitar. Jika bot musuh **bukan unit mobile standar** (mengindikasikan tower atau struktur), lokasinya dimasukkan ke queue robotTargets.
- **updateTileTargets()**: Memindai array MapInfo lokal. Setiap tile dengan PaintType.EMPTY dimasukkan ke queue tileTargets untuk dicat.
- **wander()**: Fungsi pergerakan acak. Memilih arah valid secara acak dan sebisa mungkin menghindari bergerak tepat ke arah sebaliknya dari langkah sebelumnya. Ini bertindak sebagai heuristik sederhana namun efektif untuk mencegah robot terjebak.
- **Ratio Balancing**: Tower menggunakan counter integer mentah (soldierCount, mopperCount) tanpa pembagian float untuk menentukan spawn berikutnya (soldierCount < mopperCount * 3 / 2). Pendekatan ini menjaga overhead bytecode tetap rendah sambil mempertahankan komposisi pasukan yang seimbang.

Pengujian

1. Fishybot vs Birdiebot

Fishybot (Solusi Utama) vs. Birdiebot (Alternatif 1)

- Fishybot sangat bergantung pada algoritma priority waterfall 6 tingkat. Efisiensinya berasal dari scanning O(N) pada array lokal (misalnya melakukan loop melalui robot yang terlihat untuk menemukan HP terendah). Hal ini secara efektif menjamin penyebaran sumber daya maksimum dan penolakan area yang cepat dengan secara eksplisit menangani pengisian ulang cat.
- Alternatif Birdiebot: kurang terstruktur dan bergantung pada pemeriksaan berurutan dasar serta pergerakan acak. Walaupun secara komputasi sangat murah (sangat efisien), efektivitasnya dalam strategi makro kurang karena pergerakan acak (O(1) pilihan arah) menyebabkan cakupan unit yang tumpang tindih dan distribusi cat yang tidak efisien.

2. Fishybot vs Spammer

3. Birdiebot vs Spammer

BAB 5
Kesimpulan dan Saran

1. Kesimpulan

Dalam pengerjaan Battlecode dengan algoritma Greedy, implementasi kode cukup bergantung pada musuh, lingkungan, dan rng seed. Algoritma greedy berfokus pada mengambil suatu heuristik yang dianggap optimal. Prioritas dari heuristik ini cenderung sama dan tidak dinamis terhadap perubahan yang dilakukan oleh musuh. Algoritma greedy bagus diimplementasikan untuk environment yang mudah ditebak. Misal pada Battlecode ini, kami mencari heuristik yang menghasilkan paint paling banyak. Heuristik tersebut antara lain membatasi perilaku tower dan perilaku robot. Tower akan mencoba menghancurkan dan memberitahu robot opsi terbaik yang ada. Robot akan berusaha menyebarkan paint dan membangun tower sebanyak mungkin. Algoritma greedy dapat diimplementasikan dalam pewarnaan petak, *pathfinding*, dan *spawn rate* robot.

2. Saran

Srategi algoritma menggunakan greedy memang tidak menjamin solusi yang optimal terlebih pada kasus kompleks pada game Battlecode 2025 ini. Meskipun begitu, algoritma greedy dapat dioptimasi dengan mengabstraksi dan membagi persoalan pada subpersoalan yang lebih kecil dan spesifik.

Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

13524099

Muhammad Akmal

13524047

Muhammad Jordan Ferimeison

13524025

Moh. Hafizh Irham Perdana

Plis we really need time kak :)

I really want this to be good, but OH MY GOD WAKTUNYA KURANG 😭😭😭